

Práctica de Laboratorio: CNN vs MLP con Intel Image Classification

1. Dataset

En esta práctica se utilizará el dataset **Intel Image Classification**, ampliamente utilizado en problemas de clasificación de imágenes y visión computacional.

El dataset contiene imágenes RGB de escenas naturales clasificadas en las siguientes categorías:

- buildings
- forest
- glacier
- mountain
- sea
- street

Las imágenes presentan variaciones de iluminación, perspectiva, textura y complejidad visual, lo que convierte este problema en un escenario mucho más desafiante que datasets simples como MNIST o Fashion MNIST.

Disponible en el siguiente enlace:

<https://www.kaggle.com/datasets/puneet6060/intel-image-classification>

1.1. Carga y exploración del dataset

Utilizar TensorFlow/Keras para cargar y preparar el dataset a partir de directorios de imágenes.

Responder las siguientes preguntas:

1. ¿Cuántas imágenes existen en entrenamiento, validación y prueba?
2. ¿Cuál es la resolución original de las imágenes?
3. ¿Por qué este dataset es más complejo que Fashion MNIST?
4. Identificar ejemplos donde dos clases parezcan visualmente similares.
5. ¿Qué clases parecen más difíciles de diferenciar?
6. ¿Qué impacto tiene trabajar con tres canales de color?
7. ¿Qué dificultades puede presentar este dataset para una MLP?

2. Preprocesamiento

Realizar el preprocesamiento necesario antes del entrenamiento de modelos.

1. Redimensionar todas las imágenes a una resolución uniforme.
2. Comparar el impacto de utilizar:
 - 64×64
 - 128×128
 - 150×150
3. Normalizar las imágenes en el rango $[0, 1]$.
4. Explicar por qué la normalización es importante en Deep Learning.
5. Preparar una versión flatten del dataset para utilizarla posteriormente en una MLP.
6. ¿Por qué una CNN no requiere flatten al inicio?
7. Analizar el impacto de incrementar la resolución de entrada sobre:
 - Tiempo de entrenamiento
 - Uso de memoria
 - Accuracy

3. Implementación de una MLP

Usted ya conoce la implementación de redes MLP utilizando Keras. En esta sección deberá construir una MLP base para clasificación de imágenes utilizando el dataset Intel Image Classification.

La arquitectura mínima sugerida es:

$$Input \rightarrow 512 \rightarrow 256 \rightarrow 128 \rightarrow 6$$

Considere lo siguiente:

- Utilizar capas Dense.
- Utilizar ReLU en capas ocultas.
- Utilizar Softmax en la capa de salida.
- Utilizar categorical_crossentropy como función de pérdida.
- Entrenar durante al menos 10 épocas.
- Registrar accuracy y loss de entrenamiento y validación.

Finalmente, responder:

1. ¿Por qué una MLP escala mal en imágenes complejas?
2. ¿Qué evidencia de overfitting puede observarse?
3. ¿Cómo afecta el incremento de resolución al número de parámetros?
4. ¿Qué impacto tiene esto sobre el costo computacional?

4. Implementación de una CNN

Construir una red convolucional utilizando Keras para resolver el mismo problema de clasificación.

La arquitectura mínima sugerida debe contener:

-
- Al menos tres capas convolucionales.
 - Función de activación ReLU.
 - Capas MaxPooling.
 - Una capa Flatten antes de las capas densas finales.
 - Al menos una capa Dense final.
 - Softmax como capa de salida.
 - Dropout para reducir overfitting.

Además:

1. Explicar la función de las capas convolucionales.
2. Explicar qué tipo de patrones aprende una CNN.
3. Explicar la función de MaxPooling.
4. ¿Por qué las CNN preservan mejor la estructura espacial?
5. ¿Qué representan los filtros convolucionales?
6. ¿Qué impacto tiene el tamaño del kernel?
7. ¿Qué impacto tiene incrementar el número de filtros?
8. ¿Por qué una CNN suele obtener mejor accuracy que una MLP en imágenes?
9. ¿Qué impacto tiene agregar más capas convolucionales?
10. ¿Qué relación existe entre profundidad y capacidad de representación?

5. Entrenamiento y Evaluación

Entrenar ambos modelos y comparar su desempeño.

1. Comparar accuracy de entrenamiento y prueba entre:
 - MLP
 - CNN

-
2. Comparar loss final de ambos modelos.
 3. Comparar tiempo de entrenamiento.
 4. Comparar número de parámetros entrenables.
 5. Generar e interpretar la matriz de confusión para ambos modelos.
 6. Identificar:
 - Clases con mayor confusión
 - Clases con mejor desempeño
 7. Mostrar ejemplos correctamente clasificados.
 8. Mostrar ejemplos incorrectamente clasificados.
 9. Comparar curvas de entrenamiento y validación.
 10. Identificar evidencia de:
 - Overfitting
 - Underfitting

6. Optimización del Modelo

Aplicar al menos dos mejoras sobre la CNN base:

- Incrementar número de filtros
- Agregar más capas convolucionales
- Incorporar Dropout
- Incorporar BatchNormalization
- Cambiar optimizer
- Modificar batch size
- Aplicar Data Augmentation
- Modificar learning rate

Comparar los resultados utilizando la siguiente tabla:

Modelo	Accuracy	Tiempo de entrenamiento
MLP Base		
CNN Base		
CNN Mejorada		

Responder:

1. ¿Qué mejora produjo mayor impacto?
2. ¿Qué técnicas ayudaron a reducir overfitting?
3. ¿Qué sucede si el modelo se vuelve demasiado profundo?
4. ¿Qué relación existe entre complejidad de la arquitectura y costo computacional?
5. ¿Qué ventajas tiene Data Augmentation?
6. ¿Qué impacto tuvo BatchNormalization?
7. ¿Qué impacto tuvo modificar el learning rate?
8. ¿Qué configuración obtuvo el mejor balance entre accuracy y costo computacional?

7. Actividad de Investigación

Investigar una arquitectura CNN moderna distinta a la implementada en laboratorio:

- VGG
- ResNet
- MobileNet
- EfficientNet

Elaborar una breve comparación considerando:

- Arquitectura general
- Número de parámetros
- Ventajas principales

-
- Costo computacional
 - Accuracy reportado
 - Aplicaciones reales